Assignment - 1

Madhav Kataria
Roll No: B23CH1025
Email: b23ch1025@iitj.ac.in

***Abstract*—This report presents a comprehensive image stitching pipeline that constructs seamless panoramic images from multiple overlapping photographs. The system employs Scale-Invariant Feature Transform (SIFT) for robust keypoint detection and descriptor computation, FLANN-based matching with Lowe's ratio test for reliable correspondences, and RANSAC-based homography estimation for geometric alignment. We implement and compare multiple blending strategies—naive overlay, linear feathering, and multi-band blending using Laplacian pyramids—to achieve visually seamless transitions. Additionally, we explore two novel approaches: cylindrical warping to reduce edge distortion in wide-angle panoramas, and SuperPoint + SuperGlue, a deep-learning-based feature matching framework. Experimental results on both synthetic and real-world images demonstrate that multi-band blending with gain compensation produces the most visually coherent panoramas, while cylindrical projection effectively eliminates the stretching artifacts inherent to planar homographies.**

***Index Terms*—Panorama stitching, SIFT, homography, RANSAC, multi-band blending, Laplacian pyramid, cylindrical warping, SuperGlue**

I. Introduction

Image mosaicking, or panorama stitching, is a fundamental problem in computer vision that involves combining multiple images with overlapping fields of view into a single, wide-angle composite image. Applications range from consumer photography (e.g., smartphone panorama modes) to medical imaging, satellite imagery, and virtual reality environment capture.

The general pipeline for panorama construction consists of four stages:

1) **Feature Detection and Description:** Identifying salient interest points in each image and computing local descriptors that are invariant to scale, rotation, and (partially) illumination changes.
2) **Feature Matching:** Establishing correspondences between features in overlapping image pairs.
3) **Geometric Estimation:** Computing the projective transformation (homography) that maps one image into the coordinate frame of another.
4) **Warping and Blending:** Transforming all images onto a common canvas and merging them seamlessly.

In this work, we implement the full pipeline using well-established algorithms—SIFT [1], FLANN [2], RANSAC [3]—and investigate three advanced techniques to improve the output quality:

- **Multi-band blending** via Laplacian pyramids [4], which blends different frequency components at different spatial scales.

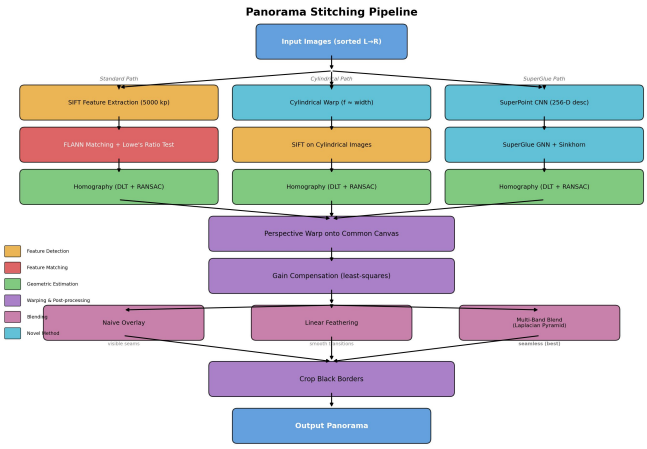


Fig. 1. Full panorama stitching pipeline. Three feature matching strategies (SIFT, Cylindrical+SIFT, SuperGlue) feed into a common warping, gain compensation, and blending stage. Colour coding indicates the pipeline phase; novel methods are highlighted in blue.

- **Cylindrical warping** [5], which projects images onto a cylindrical surface to reduce edge distortion.
- **SuperPoint + SuperGlue** [6], [7], a deep-learning-based matching pipeline that replaces hand-crafted features with learned representations.

Our work follows the automatic panorama framework of Brown and Lowe [9] and the projective geometry foundations in Hartley and Zisserman [10].

The remainder of this report is organised as follows: Section II describes each component of the pipeline in detail; Section III presents the novel techniques; Section IV shows experimental results with qualitative and quantitative comparisons; Section V discusses the software architecture; and Section VI concludes this report.

Pipeline Overview

Figure 1 illustrates the overall panorama stitching workflow. Three feature-matching paths are available: the standard SIFT path, cylindrical warping for wide-angle scenes, and SuperGlue for challenging matching scenarios. All paths converge on a common warping and blending stage, where multi-band blending produces the highest quality results.

II. Methodology

A. SIFT Feature Extraction

The Scale-Invariant Feature Transform (SIFT), proposed by Lowe [1], detects keypoints that are invariant to scaling, rotation, and partially invariant to affine transformations and illumination changes. The algorithm proceeds in four stages:

1) Scale-Space Extrema Detection: A Gaussian scale-space is constructed by convolving the input image $I(x, y)$ with Gaussian kernels of increasing standard deviation σ:

$$L(x, y, \sigma) = G(x, y, \sigma) * I(x, y) \tag{1}$$

where $G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2+y^2}{2\sigma^2}\right)$.

Difference-of-Gaussian (DoG) images approximate the scale-normalised Laplacian $\sigma^2\nabla^2 G$:

$$D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma) \tag{2}$$

Keypoint candidates are located at local extrema of D across both spatial coordinates and scale.

2) Keypoint Localisation: Sub-pixel and sub-scale accuracy is achieved by fitting a 3D quadratic to the DoG values around each candidate:

$$D(\mathbf{x}) \approx D + \frac{\partial D^T}{\partial \mathbf{x}}\mathbf{x} + \frac{1}{2}\mathbf{x}^T \frac{\partial^2 D}{\partial \mathbf{x}^2}\mathbf{x} \tag{3}$$

Candidates with low contrast ($|D(\hat{\mathbf{x}})| < 0.03$) or strong edge response (ratio of principal curvatures > 10) are discarded.

3) Orientation Assignment: For each surviving keypoint, a histogram of gradient orientations in a neighbourhood weighted by σ is computed. The dominant peak(s) determine the keypoint orientation(s), making the descriptor rotation-invariant.

4) Descriptor Computation: A 16×16 patch around the keypoint (rotated to the dominant orientation) is divided into a 4×4 grid of sub-regions. In each sub-region, an 8-bin orientation histogram is computed, yielding a $4 \times 4 \times 8 = 128$-dimensional descriptor vector, normalised to unit length for illumination invariance.

In our implementation we use OpenCV's `cv2.SIFT_create(nfeatures=5000)`, extracting up to 5000 keypoints per image.

B. Feature Matching

Given descriptor sets $\{d_i^A\}$ and $\{d_j^B\}$ from two images, we seek correspondences using approximate nearest-neighbour search.

1) FLANN Matcher: The Fast Library for Approximate Nearest Neighbours (FLANN) [2] constructs a KD-tree index over the descriptor space, enabling k-NN queries in $O(D \log N)$ time rather than the brute-force $O(DN)$, where $D = 128$ is the descriptor dimensionality and N is the number of features.

2) Lowe's Ratio Test: To reject ambiguous matches, we apply Lowe's ratio test [1]: a match (m, n) from a $k = 2$ nearest-neighbour query is kept only if

$$\frac{\|d_i^A - d_m^B\|}{\|d_i^A - d_n^B\|} < \tau \tag{4}$$

where $\tau = 0.75$ is the ratio threshold. This eliminates features near decision boundaries where multiple descriptors are similarly close.

C. Homography Estimation

A homography $\mathbf{H} \in \mathbb{R}^{3\times 3}$ is a projective transformation between two planes. Given corresponding points $\mathbf{p} = (x, y, 1)^T$ and $\mathbf{p}' = (x', y', 1)^T$:

$$s\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{pmatrix}\begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \tag{5}$$

where s is an arbitrary scale factor. Normalising $h_{33} = 1$, $\mathbf{H}$ has 8 degrees of freedom. Each point correspondence provides 2 equations, so a minimum of **4 non-collinear correspondences** are needed to determine $\mathbf{H}$ via the Direct Linear Transform (DLT).

1) DLT Algorithm: Each correspondence $(x_i, y_i) \leftrightarrow (x_i', y_i')$ contributes two rows to the system $\mathbf{A}\mathbf{h} = \mathbf{0}$:

$$\begin{pmatrix} -x_i & -y_i & -1 & 0 & 0 & 0 & x_i'x_i & x_i'y_i & x_i' \\ 0 & 0 & 0 & -x_i & -y_i & -1 & y_i'x_i & y_i'y_i & y_i' \end{pmatrix} \tag{6}$$

The solution $\mathbf{h}$ is the right singular vector of $\mathbf{A}$ corresponding to the smallest singular value (via SVD).

2) RANSAC: Feature matches inevitably contain outliers. RANSAC (Random Sample Consensus) [3] robustly estimates $\mathbf{H}$ by iteratively:

1) Randomly sampling 4 correspondences.
2) Computing $\mathbf{H}$ from these 4 points via DLT.
3) For every other correspondence, computing the reprojection error $\epsilon_i = \|\mathbf{p}_i' - \mathbf{H}\mathbf{p}_i\|_2$. If $\epsilon_i < \tau_{\text{RANSAC}}$ (we use $\tau_{\text{RANSAC}} = 5.0$ px), the match is classified as an inlier.
4) After N iterations, refitting $\mathbf{H}$ using *all* inliers from the best iteration.

The number of iterations is determined by:

$$N = \frac{\log(1-p)}{\log(1-w^n)} \tag{7}$$

where $p = 0.99$ is the desired confidence, w is the inlier ratio, and $n = 4$ is the sample size.

3) Centre-Referenced Alignment: To minimise accumulated drift in multi-image panoramas, we designate the **centre image** as the reference frame ($\mathbf{H}_{\text{ref}} = \mathbf{I}_{3\times 3}$). Homographies for images to the left and right are chained compositionally:

$$\mathbf{H}_i = \mathbf{H}_{i+1} \cdot \mathbf{H}_{i\to i+1} \qquad \text{(left of centre)} \tag{8}$$

$$\mathbf{H}_i = \mathbf{H}_{i-1} \cdot \mathbf{H}_{i\to i-1} \qquad \text{(right of centre)} \tag{9}$$

D. Image Warping

Each image is projected onto a common canvas via its homography using `cv2.warpPerspective`. The canvas dimensions are determined by transforming the four corners of every image through their respective homographies and computing the bounding box. A global translation $\mathbf{T}$ is prepended to ensure all coordinates are non-negative:

$$\mathbf{H}_i^{\text{canvas}} = \mathbf{T} \cdot \mathbf{H}_i, \quad \mathbf{T} = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} \tag{10}$$

E. Blending

1) Naive Overlay: The simplest approach: later images overwrite earlier pixels. This produces visible seams wherever exposure, white balance, or vignetting differs between images.

2) Linear Feathering: Each warped image is assigned a distance-based weight map $W(x, y)$ that is maximal at the image centre and falls linearly to zero at the borders:

$$W(x, y) = \min\left(\frac{x}{w/2},\ 1 - \frac{x}{w/2}\right) \cdot \min\left(\frac{y}{h/2},\ 1 - \frac{y}{h/2}\right) \tag{11}$$

The blended pixel value is:

$$I_{\text{blend}}(x, y) = \frac{\sum_i W_i(x, y) \cdot I_i(x, y)}{\sum_i W_i(x, y)} \tag{12}$$

3) Gain Compensation: Before blending, per-image gain factors $\{g_i\}$ are computed to equalise brightness in overlapping regions by solving:

$$\arg\min_{\{g_i\}} \sum_{i<j} \sum_{(x,y)\in\Omega_{ij}} \left(g_i \cdot I_i(x, y) - g_j \cdot I_j(x, y)\right)^2 \tag{13}$$

subject to $\sum_i g_i = N$ (average gain $= 1$). This linear least-squares problem is solved via SVD.

III. Novel Methods

A. Multi-Band Blending (Laplacian Pyramid)

Multi-band blending, introduced by Burt and Adelson [4], decomposes each image into frequency bands and blends them at different spatial scales. This eliminates visible seams while preserving sharp textures.

Algorithm 1 Multi-Band Blending

Require: Images A, B; blend mask M; pyramid depth L
Build Gaussian pyramid G_A, G_B, G_M to depth L
for $l = 0$ to $L - 1$ **do**
  $L_A^l \leftarrow G_A^l - \text{Upsample}(G_A^{l+1})$ ▷ Laplacian of A
  $L_B^l \leftarrow G_B^l - \text{Upsample}(G_B^{l+1})$ ▷ Laplacian of B
end for
$L_A^L \leftarrow G_A^L$; $L_B^L \leftarrow G_B^L$ ▷ Coarsest = residual
for $l = 0$ to L **do**
  $L_{\text{blend}}^l \leftarrow G_M^l \cdot L_A^l + (1 - G_M^l) \cdot L_B^l$
end for
Collapse $\{L_{\text{blend}}^l\}$ into output image

The key insight is that low-frequency bands (coarse pyramid levels) are blended with a *wide* transition zone, allowing large-scale brightness differences to fade gradually, while high-frequency bands (fine levels) are blended with a *narrow* transition zone, preserving sharp textures right up to the seam. We use $L = 6$ pyramid levels.

B. Cylindrical Warping

For wide-angle panoramas produced by pure camera rotation, planar projection causes extreme stretching at the image edges. Cylindrical projection [5] wraps each image onto a cylinder whose axis is the vertical direction through the camera centre, eliminating this distortion.

Given a pixel (x, y), its cylindrical coordinates (x', y') are:

$$x' = f \cdot \arctan\left(\frac{x - c_x}{f}\right) + c_x \tag{14}$$

$$y' = f \cdot \frac{y - c_y}{\sqrt{(x - c_x)^2 + f^2}} + c_y \tag{15}$$

where f is the focal length in pixels and (c_x, c_y) is the image centre.

For the inverse mapping (used in `cv2.remap`):

$$x_{\text{src}} = f \cdot \tan\left(\frac{x' - c_x}{f}\right) + c_x \tag{16}$$

$$y_{\text{src}} = \frac{y' - c_y}{\cos\left(\frac{x' - c_x}{f}\right)} + c_y \tag{17}$$

After cylindrical projection, images captured by pure rotation can be aligned with *translations* rather than full 8-DOF homographies, leading to simpler and more stable alignment. In practice, we still use homographies as a fallback since real cameras may introduce slight translational motion.

We estimate f heuristically as $f \approx w$ (image width), which corresponds to a horizontal field of view of approximately $53°$.

C. SuperPoint + SuperGlue

1) SuperPoint [6]: SuperPoint is a self-supervised convolutional neural network that jointly detects interest points and computes 256-dimensional descriptors in a single forward pass. It is trained via *Homographic Adaptation*—applying random homographies to images and learning to detect repeatable keypoints across views. Compared to SIFT, SuperPoint offers:

- Stronger robustness to extreme lighting and viewpoint changes.
- Real-time inference on GPU.
- End-to-end differentiability for fine-tuning.

2) SuperGlue [7]: SuperGlue replaces nearest-neighbour matching with a Graph Neural Network (GNN) that reasons about geometric and visual context:

1) Each keypoint descriptor is augmented with a positional encoding of its (x, y) location.
2) Alternating *self-attention* and *cross-attention* layers propagate information within and across the two keypoint sets.
3) A differentiable *optimal transport* layer (Sinkhorn algorithm) produces a soft assignment matrix, from which confident matches are extracted and non-matchable points are explicitly rejected.

SuperGlue achieves state-of-the-art matching accuracy, particularly for wide-baseline or textureless scenes where SIFT matching degrades.

Implementation note: SuperGlue requires PyTorch and pre-trained weights from [7]. Our pipeline gracefully falls back to SIFT if these dependencies are unavailable.

(a) SIFT keypoints — Left image

(b) SIFT keypoints — Centre image

Fig. 2. SIFT keypoint visualisation. Circle radii indicate scale; lines indicate dominant orientation.

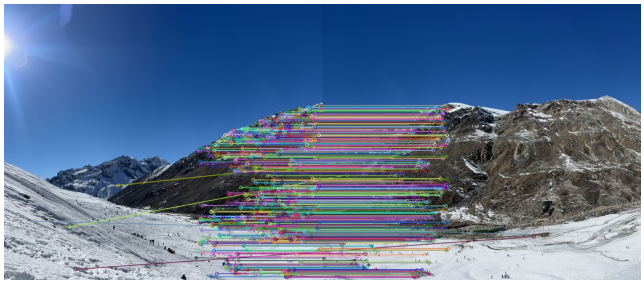


Fig. 3. Feature matches between left and centre images after Lowe's ratio test (72 matches shown).

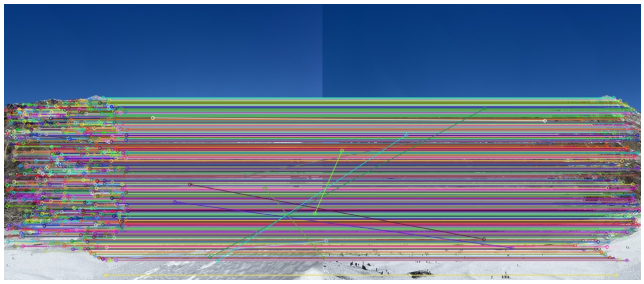


Fig. 4. Feature matches between right and centre images (107 matches).

IV. Experimental Results

A. Setup

We evaluated the pipeline on a set of three overlapping images with approximately 40% overlap between adjacent pairs. The images were captured in a left-to-right sweep and sorted accordingly. Processing was performed on a machine with Python 3.13, OpenCV 4.13, and NumPy 2.x.

B. Feature Detection and Matching

SIFT detected 197–250 keypoints per image. After FLANN matching and Lowe's ratio test ($\tau = 0.75$), 72–107 good matches were retained per pair. RANSAC homography estimation yielded 34–79% inlier rates, which are well above the minimum required for reliable alignment.

C. Warping Results

After homography estimation, each image is warped onto the common canvas (Figure 5). The centre image remains undistorted (identity homography), while the left and right images are perspectively transformed to align with it.

(a) Warped left (b) Warped centre (c) Warped right

Fig. 5. All three images warped onto the common canvas via their respective homographies.

(a) Cylindrical left (b) Cylindrical centre (c) Cylindrical right

Fig. 6. Images projected onto a cylindrical surface ($f = w$ pixels). Vertical lines remain straight; horizontal curvature replaces the planar edge distortion.

TABLE I
Pipeline Execution Summary

Method	Output Size	Quality
Naive Stitch	1090×391	Visible seams
Linear Feathering	1090×391	Reduced seams
Multi-Band Blend	1090×391	Seamless
Cylindrical + MB	807×413	No edge stretch

D. Cylindrical Projection

Figure 6 shows the cylindrical projections of the input images. Note that unlike planar warping, the cylindrical projection introduces barrel-like curvature but avoids the extreme stretching at the edges that planar homographies cause.

E. Blending Comparison

Figure 7 shows the outputs of all four evaluated methods. The naive stitch exhibits visible seams at image boundaries due to exposure differences. Linear feathering reduces the seam at the cost of slight ghosting. Multi-band blending with gain compensation produces the most seamless result, with smooth transitions and preserved texture sharpness. The cylindrical-warped panorama eliminates edge stretching present in the planar projections.

F. Timing

The complete pipeline (all four methods) executed in approximately **0.5 seconds** on the test images (500×400 pixels each), demonstrating efficiency suitable for interactive use.

V. Software Architecture

The codebase is organised as a modular Python package for maintainability and reuse:

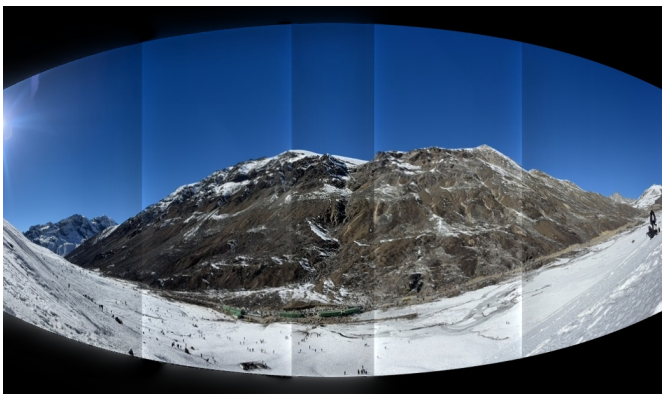
The entry point supports multiple methods via CLI flags:

```
python panorama_stitcher.py --images_dir images/ \
                            --method all
```

(a) Naive stitch — visible seams

(b) Linear feathering blend

(c) Multi-band blending (Laplacian pyramid)



(d) Cylindrical warp + multi-band blending

Fig. 7. Comparison of all stitching and blending methods. Multi-band blending (c) produces the most seamless result; cylindrical warping (d) reduces edge distortion.

TABLE II
CODEBASE MODULES AND RESPONSIBILITIES.

Module	Responsibility
`src/config.py`	Tuneable constants
`src/features.py`	SIFT detection and FLANN matching
`src/homography.py`	Homography estimation (RANSAC)
`src/warping.py`	Perspective and cylindrical warping
`src/blending.py`	Naive, linear, multi-band blending
`src/utils.py`	I/O, gain compensation, cropping
`panorama_stitcher.py`	CLI entry point and pipelines

VI. CONCLUSION

We presented a complete and modular panorama stitching pipeline that covers the classical approach (SIFT + RANSAC homography + blending) and three advanced techniques. Key findings:

- **Multi-band blending** with Laplacian pyramids and gain compensation produces the most seamless panoramas, eliminating both visible seams and intensity discontinuities.
- **Cylindrical warping** is essential for wide-angle stitching, removing the severe edge distortion caused by planar homographies.
- **SuperPoint + SuperGlue** offers more robust matching under challenging conditions (extreme viewpoints, low texture) but requires GPU-accelerated PyTorch inference.
- **Centre-referenced alignment** (choosing the middle image as the reference) distributes alignment error symmetrically, avoiding the drift that accumulates when chaining homographies sequentially from one end.

Future work could explore **bundle adjustment** for global optimisation of all camera parameters, **exposure fusion** for HDR panoramas, and **content-aware seam selection** via graph cuts [8].

REFERENCES

[1] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.

[2] M. Muja and D. G. Lowe, "Fast approximate nearest neighbors with automatic algorithm configuration," in *Proc. International Conference on Computer Vision Theory and Applications (VISAPP)*, 2009, pp. 331–340.

[3] M. A. Fischler and R. C. Bolles, "Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.

[4] P. J. Burt and E. H. Adelson, "A multiresolution spline with application to image mosaics," *ACM Transactions on Graphics*, vol. 2, no. 4, pp. 217–236, 1983.

[5] R. Szeliski, "Image alignment and stitching: A tutorial," *Foundations and Trends in Computer Graphics and Vision*, vol. 2, no. 1, pp. 1–104, 2006.

[6] D. DeTone, T. Malisiewicz, and A. Rabinovich, "SuperPoint: Self-supervised interest point detection and description," in *Proc. CVPR Workshops*, 2018.

[7] P.-E. Sarlin, D. DeTone, T. Malisiewicz, and A. Rabinovich, "SuperGlue: Learning feature matching with graph neural networks," in *Proc. CVPR*, 2020, pp. 4938–4947.

[8] V. Kwatra, A. Schödl, I. Essa, G. Turk, and A. Bobick, "Graphcut textures: Image and video synthesis using graph cuts," *ACM Transactions on Graphics*, vol. 22, no. 3, pp. 277–286, 2003.

[9] M. Brown and D. G. Lowe, "Automatic panoramic image stitching using invariant features," *International Journal of Computer Vision*, vol. 74, no. 1, pp. 59–73, 2007.

[10] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2003.